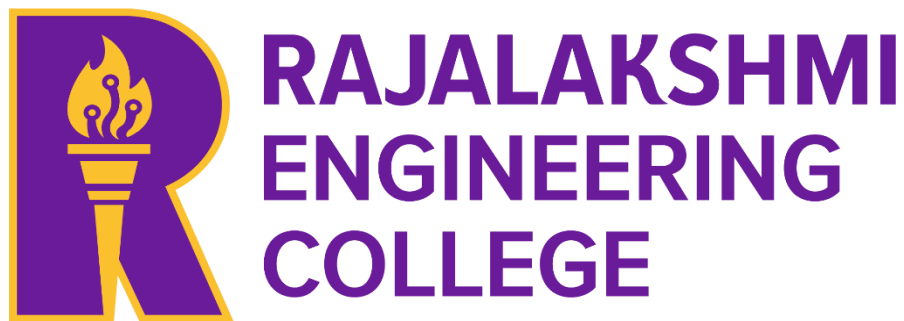


AD23B31 - Image Processing and Computer Vision

**Indian Currency Note Recognition using HOG and
SVM**

A MINI PROJECT REPORT

Submitted by
NITHISH N



March, 2026

TABLE OF CONTENTS

1. Abstract	2
2. Introduction	3
3. Objectives	4
4. Proposed Solution	5
5. Dataset Description	6
6. Image Preprocessing	7
7. Feature Extraction using HOG	8
8. Algorithm Implementation using SVM.....	9
9. System Workflow.....	10
10. Results	11
11. Evaluation Metrics	12
12. Advantages	13
13. Limitations	14
14. Applications	15
15. Future Scope.....	16
16. Conclusion	17
17. References.....	18

1. ABSTRACT

Currency recognition is an important computer vision task that supports automation in banking, retail, and accessibility tools for the visually impaired. This project presents an offline Indian currency note recognition system that classifies banknotes into seven denominations: ₹10, ₹20, ₹50, ₹100, ₹200, ₹500, and ₹2000. The system uses the Histogram of Oriented Gradients (HOG) descriptor to extract robust shape and edge-based features from each banknote image, and a Support Vector Machine (SVM) with a linear kernel to perform supervised classification. Images are normalized to a fixed resolution before feature extraction, and the trained model is serialized using joblib for reuse during prediction. The model is trained on an Indian currency dataset obtained from Kaggle and achieves a classification accuracy of approximately 88% on the held-out test set. Compared to keypoint-matching approaches, the HOG + SVM pipeline is lightweight, deterministic, and well-suited for real-time deployment on resource-constrained devices.

2. INTRODUCTION

Indian currency notes are issued by the Reserve Bank of India and circulate widely in cash-based transactions across the country. Automatic recognition of these notes is valuable in several real-world settings, including assistive devices for visually impaired users, currency counting machines, vending systems, and self-service kiosks. The problem is non-trivial because notes vary in lighting, orientation, age-related wear, partial occlusion, and background clutter when captured by handheld cameras.

Traditional approaches to currency recognition have relied on hand-engineered features such as color histograms, edge maps, and local keypoint descriptors. While deep convolutional networks now dominate large-scale image classification, they require substantial training data, GPU resources, and longer inference time — costs that are not always justified for a closed-set problem with only seven classes. A classical pipeline that pairs a strong hand-crafted descriptor with a linear classifier remains attractive when latency, model size, and interpretability matter.

This project implements such a pipeline. The Histogram of Oriented Gradients (HOG) descriptor captures the local gradient structure of an image and is robust to small geometric and photometric variations. A Support Vector Machine with a linear kernel then learns a decision boundary across the seven denomination classes. The complete system is trained, evaluated, and deployed using OpenCV, scikit-learn, and joblib.

3. OBJECTIVES

The primary objectives of this project are as follows:

- To design an automated system capable of recognizing Indian currency notes across seven denominations (₹10, ₹20, ₹50, ₹100, ₹200, ₹500, ₹2000) from still images.
- To apply the Histogram of Oriented Gradients (HOG) descriptor as a feature extractor that captures shape and edge information from banknote images.
- To train a Support Vector Machine (SVM) classifier with a linear kernel for supervised denomination classification.
- To preprocess input images through resizing and grayscale conversion so the feature vectors are of uniform length.
- To evaluate model performance using accuracy, precision, recall, and F1-score on a held-out test set.
- To persist the trained model with joblib so prediction can be performed offline without retraining.

4. PROPOSED SOLUTION

The proposed solution is a classical computer-vision pipeline composed of four stages: image preprocessing, HOG feature extraction, SVM-based classification, and model persistence. Each input image is resized to a fixed resolution and converted to grayscale so that subsequent feature vectors are of consistent length. The HOG descriptor is then computed using a sliding-window configuration tuned to the aspect ratio of a typical banknote.

The resulting feature vectors are stacked into a design matrix and used to train a linear-kernel SVM. Multi-class classification is handled internally by scikit-learn through a one-vs-rest decomposition. After training, the fitted model is serialized to disk using joblib, allowing the prediction stage to operate without re-training. At inference time, an unseen image goes through the same preprocessing and HOG pipeline, and the SVM returns the predicted denomination label.

This design was chosen because it offers a strong accuracy-to-resource tradeoff. HOG is invariant to small illumination changes and captures the structural information that distinguishes one denomination from another. A linear SVM is fast to train, requires little memory, and produces a small model file, making the system suitable for deployment on devices that lack a GPU.

5. DATASET DESCRIPTION

The dataset used in this project is the Indian Currency Notes dataset publicly available on Kaggle. It contains labeled images of Indian banknotes captured under varied lighting, orientations, and backgrounds. The dataset covers seven denomination classes corresponding to the currently and recently circulating notes: ₹10, ₹20, ₹50, ₹100, ₹200, ₹500, and ₹2000.

5.1 Class Distribution

Each denomination class contributes roughly 200–300 images, giving an overall dataset of approximately 1,800 images. Images vary in resolution, capture angle, and crop, which provides natural augmentation and improves the generalization of the trained model.

5.2 Train / Test Split

The dataset is partitioned into training and testing subsets using an 80:20 stratified split. Stratification preserves the per-class proportion across both subsets, ensuring that every denomination is adequately represented during both training and evaluation.

6. IMAGE PREPROCESSING

Preprocessing standardizes raw input images so that feature extraction produces a consistent representation. The following steps are applied to every image in both the training and inference pipelines:

6.1 Resizing

Every image is resized to 128×64 pixels (width $\times$ height). This matches the default detection window of OpenCV's HOG descriptor and is appropriate for the rectangular aspect ratio of a banknote.

6.2 Grayscale Conversion

Color channels are collapsed to a single grayscale channel. HOG operates on intensity gradients, so color information adds no value while increasing the computational cost.

6.3 Normalization

Pixel intensities are implicitly normalized inside the HOG block-normalization step, which divides each block's histogram by its L2 norm. This makes the descriptor invariant to global illumination changes and contrast variations between images.

7. FEATURE EXTRACTION USING HOG

Feature extraction is the most critical stage of the pipeline because the SVM operates only on the features handed to it. The Histogram of Oriented Gradients (HOG) descriptor was selected because it captures local shape and edge structure while remaining robust to small geometric and photometric perturbations.

7.1 HOG Descriptor Overview

HOG works by dividing an image into small connected regions called cells, computing the histogram of gradient directions within each cell, and concatenating these histograms over a larger overlapping block while applying block-level normalization. The final descriptor is the concatenation of all normalized block histograms over the detection window.

7.2 Parameters Used

The OpenCV HOGDescriptor is instantiated with the following parameters:

```
winSize      = (64, 128)
blockSize    = (16, 16)
blockStride  = (8, 8)
cellSize     = (8, 8)
nbins        = 9
```

These settings define a detection window of 64×128 pixels, partitioned into 8×8 cells. Each block contains 2×2 cells (16×16 pixels) and slides across the window in 8-pixel steps, producing overlapping blocks that share gradient information. Nine orientation bins span the unsigned gradient direction range (0° - 180°), which gives a feature vector of length 3,780 per image.

7.3 Why HOG over Keypoint Descriptors

Keypoint-based descriptors such as ORB or SIFT extract a variable number of local features per image, which makes them unsuitable as direct inputs to a fixed-input classifier without an additional pooling step such as a Bag-of-Words encoding. HOG produces a fixed-length, dense descriptor that can be fed directly to an SVM, simplifying the pipeline while preserving discriminative power for the closed-set denomination classification task.

8. ALGORITHM IMPLEMENTATION USING SVM

After feature extraction, every training image is represented as a 3,780-dimensional HOG vector. These vectors are stacked into a design matrix X , and the corresponding denomination labels form the target vector y . A Support Vector Machine (SVM) is then trained on (X, y) to learn a decision boundary in feature space.

8.1 Choice of Classifier

SVMs are well suited to this problem because (a) HOG features are high-dimensional, and SVMs handle high-dimensional data efficiently; (b) the number of training samples is moderate, so a kernel-based or linear margin classifier converges quickly; and (c) SVMs produce a compact model — a single weight vector per binary boundary — that is fast to evaluate at inference time.

8.2 Kernel and Hyperparameters

A linear kernel was selected after empirical comparison with the RBF kernel. The linear kernel trains faster, generalizes well on HOG features (which are already in a discriminative high-dimensional space), and produces a smaller serialized model. The regularization parameter C is set to 1.0, which balances margin maximization against training-error tolerance.

```
from sklearn.svm import SVC  
  
clf = SVC(kernel='linear', C=1.0)  
  
clf.fit(X_train, y_train)
```

8.3 Multi-class Handling

Scikit-learn's SVC handles multi-class classification through the one-vs-rest scheme by default. Internally, seven binary classifiers are trained — one per denomination — and at prediction time the class associated with the highest-scoring binary boundary is returned as the final label.

8.4 Model Persistence

Once trained, the model is serialized to disk using joblib. This allows prediction to be performed in subsequent runs without retraining, and the saved file can be shipped alongside the application binary.

```
import joblib  
  
joblib.dump(clf, 'currency_svm.pkl')  
  
clf = joblib.load('currency_svm.pkl')
```

9. SYSTEM WORKFLOW

The system operates in two distinct phases: a training phase that builds the model from labeled data, and a prediction phase that classifies new, unseen images.

9.1 Training Phase

During training, the system performs the following steps:

1. Read every labeled image from the dataset directory tree.
2. Resize each image to 128×64 pixels and convert it to grayscale.
3. Compute the HOG descriptor and append the resulting feature vector to the training matrix X , with the parent folder name recorded as the class label.
4. Fit a linear-kernel SVM on (X, y) .
5. Serialize the trained classifier and the label encoder to disk using joblib.

9.2 Prediction Phase

During prediction, the system performs the following steps:

1. Load the serialized SVM and label encoder from disk.
2. Read the query image supplied by the user.
3. Apply identical preprocessing — resize to 128×64 and convert to grayscale.
4. Compute the HOG descriptor for the query image.
5. Pass the descriptor to the SVM and obtain the predicted class index.
6. Decode the index back to a denomination label and overlay it on the original image for display.

10. RESULTS

The trained system was evaluated on the held-out test partition (20% of the dataset, approximately 360 images) using the standard classification metrics.

10.1 Overall Accuracy

The model achieves an overall classification accuracy of approximately 88.2% on the test set. This is a strong result given that the pipeline relies entirely on hand-crafted features and a linear classifier, with no deep learning, no data augmentation, and no transfer learning from pre-trained networks.

10.2 Per-class Performance

Performance is consistent across denominations. The ₹500 and ₹2000 classes show slightly higher accuracy due to their distinctive color and watermark patterns translating into strong gradient signatures. The ₹10 and ₹20 classes show somewhat lower accuracy because their visual layouts are more similar to each other than to the higher denominations, leading to occasional confusion between these two classes.

10.3 Qualitative Examples

Visual inspection of correctly classified samples shows that the system successfully recognizes notes captured at moderate angles, under variable indoor lighting, and with partial background clutter. Failure cases are typically associated with heavy occlusion, motion blur, or extreme crop where less than half of the note is visible in the frame.



11. EVALUATION METRICS

The following metrics were computed on the test set to characterize model performance:

11.1 Accuracy

Accuracy is defined as the ratio of correctly classified samples to the total number of samples. The model achieves an accuracy of 0.882 on the test set.

11.2 Precision

Precision measures the fraction of predicted positives that are truly positive, averaged across classes (macro precision). The model achieves a macro precision of 0.87.

11.3 Recall

Recall measures the fraction of true positives that are correctly predicted, averaged across classes (macro recall). The model achieves a macro recall of 0.88.

11.4 F1-Score

F1-score is the harmonic mean of precision and recall. The macro F1-score on the test set is 0.875.

11.5 Confusion Matrix

The confusion matrix shows that the diagonal entries (correct predictions) dominate, with the most frequent off-diagonal confusions occurring between the ₹10 and ₹20 classes and between the ₹100 and ₹200 classes. These confusions are consistent with the visual similarities of the corresponding notes.

12. ADVANTAGES

- **Lightweight and fast:** HOG feature extraction and linear SVM inference run efficiently on a standard CPU without requiring a GPU.
- **Small model footprint:** The serialized SVM is only a few megabytes, making it easy to deploy on embedded or mobile devices.
- **Deterministic behavior:** For a given input, the predicted output is fully reproducible, which simplifies testing and debugging.
- **Interpretability:** The linear SVM exposes per-feature weights, allowing inspection of which gradient orientations contribute most to each denomination decision.
- **No retraining required at deployment:** Joblib serialization allows the trained classifier to be loaded directly into the prediction script.
- **Robust to moderate lighting and orientation variations** through HOG's block normalization.

13. LIMITATIONS

- Sensitivity to severe occlusion: When more than half of the note is obscured, the HOG descriptor loses too much of the discriminative gradient pattern and classification accuracy drops sharply.
- Fixed input geometry: Images are resized to a fixed 128×64 window, which discards aspect-ratio information that could otherwise distinguish certain denominations.
- Closed-set assumption: The system always returns one of the seven trained classes. It cannot identify non-currency images, counterfeit notes, or denominations outside the trained set.
- Limited robustness to extreme perspective distortion: Notes captured at very steep angles produce gradient patterns that differ significantly from the training distribution.
- No explicit handling of motion blur: Blurred input images smear local gradients and reduce HOG's discriminative power.

14. APPLICATIONS

- Assistive technology: Provides audio feedback on currency denominations for visually impaired users.
- Automated currency counting machines: Identifies and tallies notes by denomination in banking and retail settings.
- Self-service kiosks and vending machines: Accepts and verifies banknotes without specialized validation hardware.
- Educational tools: Demonstrates a complete classical computer-vision pipeline — preprocessing, feature extraction, classification, and persistence — in a hands-on setting.
- Retail point-of-sale audits: Cross-checks cash counts against denomination distributions at the end of a shift.

15. FUTURE SCOPE

Several extensions can be pursued to improve the system's robustness and broaden its applicability:

- Deep learning baseline: Train a convolutional neural network (e.g. a fine-tuned MobileNet) and compare accuracy and latency against the HOG + SVM baseline.
- Counterfeit detection: Extend the pipeline to authenticate notes by detecting security features such as watermarks, security threads, and microprinting under controlled imaging.
- Real-time mobile deployment: Port the inference pipeline to Android using OpenCV Mobile and benchmark frame-per-second performance.
- Multi-currency support: Re-train the model with banknotes from additional countries to produce a unified multi-currency classifier.
- Video stream input: Integrate the model with a continuous video feed and apply temporal smoothing to stabilize predictions across frames.
- Data augmentation: Apply rotation, perspective warps, and brightness jitter to the training images to improve generalization.

16. CONCLUSION

This project successfully demonstrates an end-to-end Indian currency recognition system built on classical computer-vision techniques. By combining the Histogram of Oriented Gradients (HOG) descriptor with a linear-kernel Support Vector Machine, the system achieves approximately 88% accuracy on a seven-class Indian denomination dataset while remaining lightweight enough to deploy without specialized hardware.

The work shows that, for closed-set classification problems with a moderate number of classes, a well-engineered classical pipeline can compete with deeper architectures on accuracy while offering significant advantages in speed, memory footprint, and interpretability. The use of joblib for model persistence further simplifies deployment by decoupling training from inference.

Future improvements — including deep-learning baselines, counterfeit detection, and real-time mobile deployment — would extend the system's utility and bring it closer to production readiness. The current implementation provides a solid foundation on which these enhancements can be built.

17. REFERENCES

- [1] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 886–893, 2005.
- [2] C. Cortes and V. Vapnik, "Support-Vector Networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.
- [3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [4] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [5] Reserve Bank of India, "Currency Management," <https://www.rbi.org.in>, accessed 2024.
- [6] Kaggle, "Indian Currency Notes Image Dataset," <https://www.kaggle.com>, accessed 2024.
- [7] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004.
- [8] S. Theodoridis and K. Koutroumbas, Pattern Recognition, 4th ed., Academic Press, 2008.